

Text summarization using vectors typically falls under **extractive summarization** and leverages **word embeddings** or **sentence embeddings** to represent the text's semantic content. By converting text into a numerical vector space, we can use geometric measures (like distance or similarity) to identify the most important sentences.

Key Concepts

1. Vector Representation (Embeddings)

The core idea is to transform the input text (document) into vectors:

- **Word Embeddings (e.g., Word2Vec, GloVe):** Each word is mapped to a dense vector, capturing its meaning based on its context.
- **Sentence Embeddings (e.g., Doc2Vec, Sentence-BERT):** This is the more common approach for summarization. Each entire sentence is mapped to a single dense vector. Crucially, vectors of sentences with similar meanings will be **close together** in the vector space.

2. Similarity and Centrality

The effectiveness of vector-based summarization relies on two geometric concepts:

- **Similarity:** Measured using **Cosine Similarity** between two sentence vectors A and B .

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A higher cosine similarity indicates that the two sentences are semantically closer.

- **Centrality:** The importance of a sentence is determined by its semantic similarity to **all other sentences** in the document. A sentence whose vector is close to the vectors of many other sentences is considered **central** and likely contains the core topic.
-

Extractive Summarization Steps

The process for generating a summary using sentence vectors typically follows these steps:

1. Pre-processing and Segmentation

The input document is cleaned (e.g., punctuation handled, stop words potentially removed) and split into individual sentences.

2. Vectorization

Every sentence in the document is converted into a high-dimensional **sentence embedding vector** using a pre-trained model (e.g., Sentence-BERT).

3. Building the Similarity Matrix

A square matrix (where the size is $N \times N$, and N is the number of sentences) is constructed. Each cell (i, j) in the matrix stores the **cosine similarity** between sentence i and sentence j .

4. Scoring Sentences (Centrality)

To determine the score (importance) of each sentence, two common methods are used:

- **Average Similarity:** The score of a sentence is the average of its similarity scores with all other sentences in the document.
- **Graph-based Ranking (e.g., TextRank):** The similarity matrix is treated as an adjacency matrix of a graph, where sentences are nodes and similarity scores are edge weights. An algorithm similar to Google's PageRank is run to iteratively calculate the "rank" (importance) of each sentence within the document's semantic structure.

5. Summary Generation

The sentences are ranked based on their scores. The top K highest-scoring sentences are selected to form the final summary.

6. Ordering

The selected sentences are usually placed back into the summary in their **original order** of appearance in the document to ensure the summary flows logically and maintains temporal coherence.



VS Advantages of the Vector Approach

- **Semantic Awareness:** Unlike earlier methods (e.g., counting word frequency), vector-based summarization captures the actual **meaning** of the text, ensuring the selected sentences relate to the core concepts.
- **Language Independence:** Pre-trained embeddings can often work across different languages (especially multilingual models).
- **Robustness to Synonyms:** Sentences that use different words but convey the same meaning will have similar vectors, preventing similar ideas from being double-counted or missed.

The intuition behind **TextRank** is simple and powerful: **The importance of a piece of text (like a sentence or a keyword) is determined by the importance of the information linked to it, and how strongly they are connected.**

TextRank adapts the well-known **PageRank** algorithm, which ranks the importance of web pages, to the world of text.

Here is a breakdown of the core intuition:

Core Intuition: Voting and Centrality

1. PageRank Analogy (The Origin)

- **PageRank Intuition:** In the World Wide Web, a webpage is considered important if many other important pages link to it. A link from an important page (high rank) counts for more than a link from an unimportant page (low rank).
- **TextRank Adaptation:** In a document, a **sentence** is considered important if it shares many semantic or lexical connections (shared words/ideas) with other important sentences.

2. Building the Semantic Graph

- **Nodes as Ideas:** Every sentence (or keyword) in the document is treated as a **node** in a graph.
- **Edges as Connections:** A connection (an **edge**) is drawn between two nodes if the sentences/keywords are **similar** (i.e., they share common words or phrases). The strength of the connection (the **edge weight**) is the measure of their similarity.

3. The Voting Mechanism

- **Every sentence is a voter.** When two sentences are connected, they "vote" for each other.
- The **score** of a sentence is the total influence it receives from all the sentences linked to it.
- This voting is **iterative**:
 - Initially, all sentences have the same rank.
 - In each round, a sentence's rank is updated based on the ranks of its neighbors from the previous round.
 - A sentence strongly linked to a highly-ranked neighbor gains a high rank itself.

4. Centrality and Topic Maintenance

The final high-ranking sentences are those that are **central** to the document.

- A high TextRank score signifies that a sentence is closely related to a large number of other sentences in the document.
- These sentences effectively summarize the **main theme** or **core ideas** because they act as hubs of semantic connection, tying the various parts of the text together.

Summary of Intuition

The intuition is that the **most relevant sentence** for a summary is not just the one that is most different, but the one that is **most effective at connecting the ideas expressed throughout the text**. It's the **semantic hub** of the document.

Topic modeling refers to a suite of unsupervised methods used to discover the hidden, abstract "**topics**" that underlie a collection of documents (a corpus). These techniques analyze the co-occurrence of words to infer semantic structures. The primary techniques include **Latent Dirichlet Allocation (LDA)**, **Non-Negative Matrix Factorization (NMF)**, and **Neural/Embedding-based Models (like BERTopic)**.

1. Latent Dirichlet Allocation (LDA)

LDA is the most classic and widely used **generative probabilistic model** for topic modeling.

Intuition and Mechanism

- **Generative Process:** LDA assumes that every document is a mixture of various underlying **topics**, and that every topic is a mixture of various words.
- **Probability Distributions:** The model seeks to infer two key distributions from the observed data:
 1. $P(\text{word}|\text{topic})$: The probability of a particular word occurring given a specific topic. This defines the composition of each topic.
 2. $P(\text{topic}|\text{document})$: The probability of a topic occurring in a specific document. This defines the topic mixture of each document.
- **Dirichlet Prior:** LDA uses **Dirichlet priors** to smooth these distributions, ensuring that topics don't become overly dominated by a few words and that documents utilize a manageable number of topics. This is its key distinction from its predecessor, pLSA.

Implementation Detail

LDA requires the user to specify the **number of topics** (K) beforehand. It typically uses techniques like **Gibbs sampling** or **Variational Inference** to approximate the posterior distributions.

2. Non-Negative Matrix Factorization (NMF)

NMF is a **linear algebra-based** approach that relies on dimensionality reduction rather than probabilistic assumptions.

Intuition and Mechanism

- **Matrix Decomposition:** NMF takes the **Term-Document Matrix** (V)—where rows are words and columns are documents, populated by word counts or TF-IDF scores—and decomposes it into two smaller matrices:

$$V \approx WH$$

- **W (Word=Topic Matrix):** Defines the topics. Each column represents a topic and shows the weights of words belonging to that topic.
- **H (Topic-Document Matrix):** Defines document content. Each column shows the degree to which a document contains each topic.
- **Non-Negativity Constraint:** The crucial constraint is that all entries in **W** and **H** must be **non-negative** (≥ 0). This ensures that the components (topics) are purely additive and easily interpretable, as concepts are never defined by the *absence* (subtraction) of words.



Key Feature

NMF is deterministic, highly efficient, and yields easily interpretable results, though like LDA, it generally requires setting the number of latent components (topics) K .

3. Neural and Embedding-Based Models (BERTopic)

Modern topic modeling leverages state-of-the-art **Transformer-based language models** to capture deep, contextual meaning. **BERTopic** is a prominent example.

Intuition and Mechanism

- **Semantic Embeddings:** Unlike LDA and NMF, which rely only on word counts, BERTopic first uses a pre-trained model (like BERT or Sentence-BERT) to convert each document into a single, high-quality **semantic vector (embedding)**.
- **Clustering:** Documents with similar meanings will have vectors that are close together in the vector space. The algorithm applies a **density-based clustering** technique (like HDBSCAN) to group these similar document embeddings. Each resulting cluster is defined as a **topic**.
- **Topic Representation:** It uses a modified TF-IDF procedure called **c-TF-IDF** (class-based TF-IDF) to find the words that best distinguish one cluster (topic) from all others.

Advantages

- **Contextual Coherence:** Topics are highly coherent because they are derived from contextual embeddings, not just word co-occurrence counts.
- **Automatic Topic Number:** HDBSCAN automatically determines the optimal number of clusters/topics, removing the need for manual parameter tuning for K .

Comparison and Suitability

Technique	Underlying Principle	Input Data	Key Requirement	Topic Coherence
LDA	Probabilistic (Generative)	Term-Frequency Matrix	Pre-set Topic Number (K)	Good, but depends on K
NMF	Linear Algebra (Decomposition)	Term-Document Matrix	Pre-set Topic Number (K)	Good, highly interpretable
BERTopic	Neural Embeddings + Clustering	Document Embeddings	No K required (automatic)	High (Contextual)



For academic study on sparse data, **LDA** remains a standard benchmark. For high-quality, semantically rich, and context-aware topics on modern text data, **BERTopic** is the current state-of-the-art.

Latent Semantic Analysis (LSA) / Latent Semantic Indexing (LSI)

Latent Semantic Analysis (LSA), often referred to as **Latent Semantic Indexing (LSI)** when applied to information retrieval, is a technique in natural language processing (NLP) and information retrieval that analyzes relationships between a set of documents and the terms they contain by producing a set of concepts related to the documents and terms.

The core idea is that words that appear in similar contexts (documents) tend to have similar meanings, and by mapping documents and terms to a lower-dimensional **latent semantic space**, LSA can capture and exploit these underlying semantic relationships.

Singular Value Decomposition (SVD) Intuition

Singular Value Decomposition (SVD) is the mathematical engine behind LSA. It is a powerful matrix factorization technique that decomposes a matrix into three constituent matrices.

The Intuition

Imagine you have a large dataset (represented as a matrix **A**). SVD helps you find the most important, independent **"factors"** or **"components"** that explain most of the variation in the original data.

SVD decomposes the original **Term-Document Matrix A** (where rows are terms/words, columns are documents, and entries are frequencies) into three matrices:

$$\mathbf{A}_{m \times n} = \mathbf{U}_{m \times m} \mathbf{\Sigma}_{m \times n} \mathbf{V}_{n \times n}^T$$

1. **$\mathbf{U}$ (Left Singular Vectors):** This matrix represents the relationship between the **terms** and the **latent concepts/topics**.
2. **Σ (Singular Values):** This is a diagonal matrix containing the **singular values** ($\sigma_1, \sigma_2, \dots$) ordered from largest to smallest. These values represent the **strength** or **importance** of each latent concept. Larger singular values correspond to more important concepts.
3. **$\mathbf{V}^T$ (Right Singular Vectors):** This matrix represents the relationship between the **documents** and the **latent concepts/topics**.

Dimensionality Reduction (The Key)

The magic happens when we perform a **truncated SVD**. Since the singular values are ranked by importance, we can discard the smallest ones (the noise) and keep only the top k largest singular values and their corresponding columns in $\mathbf{U}$ and $\mathbf{V}^T$.

By doing this, we create a new, lower-rank matrix $\mathbf{A}_k$ that is the best approximation of the original matrix $\mathbf{A}$.

$$\mathbf{A}_{m \times n} \approx \mathbf{U}_{m \times k} \Sigma_{k \times k} \mathbf{V}_{k \times n}^T$$

This process projects the high-dimensional data (millions of words across thousands of documents) into a much smaller, **k -dimensional latent semantic space**.

LSA / LSI: Applying SVD to NLP

When SVD is applied to the Term-Document Matrix $\mathbf{A}$, the resulting components gain semantic meaning:

1. **Term Representation:** A row in the truncated $\mathbf{U}_k$ matrix represents a **term**'s coordinates in the k -dimensional semantic space. Terms that are close together in this space are semantically similar, even if they never appeared together in the same document (e.g., "car" and "automobile").
2. **Document Representation:** A column in the truncated $\mathbf{V}_k^T$ matrix represents a **document**'s coordinates in the k -dimensional semantic space. Documents that are close together are semantically similar.
3. **Latent Concepts:** The k dimensions themselves are the **latent topics** or **concepts**. A document is represented as a weighted combination of these concepts, and a word is represented as a weighted combination of these concepts.

Advantages of LSA/LSI

- **Synonymy:** It handles synonymy (different words having the same meaning) by grouping them close together in the latent space.
- **Polysemy:** It can partially address polysemy (one word having multiple meanings) by placing the word between the different concepts it represents.
- **Noise Reduction:** By removing the smallest singular values, LSA effectively filters out noise and low-frequency variations, focusing on the strong, underlying semantic structure.

Latent Semantic Analysis / Latent Semantic Indexing in Python

In Python, LSA is typically implemented using the `scikit-learn` library, often using `TruncatedSVD` on the TF-IDF representation of the text.

Conceptual Steps

1. **Input Data:** A collection of documents (the corpus).
2. **Vectorization (Term-Document Matrix):** Convert the text into a numerical matrix. The **TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer** is commonly used, as it weights important, non-common words higher.
3. **Apply Truncated SVD:** Apply the SVD algorithm to the resulting TF-IDF matrix, specifying the desired number of latent topics (k).
4. **Analysis:** Examine the resulting matrices to interpret the topics and use the document vectors for downstream tasks.